

Optimización de rendimiento en juegos

Como desarrollador, lo primero es **medir y perfilar** el juego antes de optimizar. Define un **presupuesto de tiempo por frame** (por ejemplo, 16.66 ms para 60 FPS o 33.33 ms para 30 FPS) y mide el *tiempo de fotograma*, no solo los FPS ¹ ². Por ejemplo, bajar de 60 a 56.25 FPS supone solo ~1.1 ms extra por frame ¹ ², un cambio imperceptible, pero bajo presupuesto ya causa **tirones**. Como muestra el gráfico, es mejor enfocarse en el tiempo (ms) que en los FPS absolutos ¹ ². Además:

- **Perfila desde el inicio** y con frecuencia ³. Usa herramientas integradas (Unity Profiler, Unreal Insights, Godot Profiler, etc.) y externas (Visual Studio Profiler, RenderDoc, NVIDIA Nsight, AMD uProf/RGP, Perfetto en móviles, etc.) para identificar cuellos de botella de CPU vs GPU ⁴ ⁵. Mide antes y después de cada cambio importante ³.
- **Distingue si el cuello es de CPU o GPU** ⁴ ⁵. Por ejemplo, en Unreal usar `stat unit` (divide tiempos CPU/GPU) y `stat gpu` (tiempos GPU, draw calls, triángulos) ⁵. En Unity, el Profiler y el Frame Debugger ayudan a ver draw calls innecesarios ⁶ ⁷. En Godot, la sección de rendimiento señala que un profiler rápido revelará qué funciones consumen más tiempo ⁸.

Fig. Gráfico que ilustra por qué medir en *tiempo de fotograma* (ms) es más significativo que en FPS ¹ ². Un mismo golpe de ~1.11 ms extra (de 16.66 a 17.77 ms por frame) apenas baja los FPS (60 → 56), pero sí rompe el presupuesto si está al límite ¹ ².

Optimización en CPU

- **Identificar cuellos de botella:** usa el profiler para hallar qué funciones o procesos consumen más CPU ⁸. En Godot advierten que casi siempre una función destacada es el cuello de botella principal ⁸. Optimiza esas partes; no pierdas tiempo en código que aporta poco.
- **Reducir trabajo por frame:** evita cálculos innecesarios en cada `Update`. Por ejemplo, no actualices la IA o la física de todos los NPC cada fotograma; hazlo en pausas, cada X segundos o con *spatial partitioning*. Desactiva lógicas de objetos lejanos o inactivos. Como regla general: “*el mejor cálculo es el que no se hace*”.
- **Evitar gastos de llamadas a API:** en Unity, llamadas frecuentes como `GetComponent` o `Find` por frame son costosas. Cachea referencias y usa `sharedMaterial` en lugar de `material` para no duplicar materiales y romper batches ⁹. Minimiza la creación y destrucción de GameObjects. En lugar de `Instantiate` / `Destroy`, usa **object pooling**: crea un conjunto de objetos al inicio (inactive) y réutilízalos ¹⁰. Esto reduce drásticamente el trabajo del Garbage Collector y evita picos de CPU.

```
// Ejemplo sencillo de *object pooling* en Unity (C#):
public class Pooler : MonoBehaviour {
    public GameObject prefab;
    public int size = 20;
    private Queue<GameObject> pool = new Queue<GameObject>();

    void Awake() {
        for(int i=0; i<size; i++) {
            GameObject obj = Instantiate(prefab);
```

```

        obj.SetActive(false);
        pool.Enqueue(obj);
    }
}

// Obtiene un objeto del pool (activa el más antiguo o crea uno nuevo)
public GameObject GetFromPool() {
    if (pool.Count > 0) {
        GameObject obj = pool.Dequeue();
        obj.SetActive(true);
        return obj;
    }
    // Opcional: si no hay disponibles, instanciar uno nuevo
    GameObject extra = Instantiate(prefab);
    return extra;
}

// Devuelve un objeto al pool (lo desactiva y encola)
public void ReturnToPool(GameObject obj) {
    obj.SetActive(false);
    pool.Enqueue(obj);
}
}

```

- **Multihilo y Jobs:** si tu motor lo permite (Unity C# Job System, ECS, hilos nativos, etc.), pasa cálculos costosos a otros hilos para liberar el hilo principal. GPUOpen menciona que el comando `stat unit` en Unreal ayuda a ver si el motor está limitado en el hilo de render o en hilos worker ⁵, lo que guía a repartir trabajo.
- **Uso eficiente de la memoria cache:** los Godot docs recuerdan que un mismo código puede medir distinto según si los datos están en cache de CPU ¹¹. Accede a arreglos de forma lineal y evita saltos de memoria grandes. Esto mejora drásticamente la velocidad de la CPU al cargar datos secuencialmente en caché.

Optimización en GPU (gráficos)

- **Minimizar draw calls:** cada *draw call* tiene overhead de CPU (cambios de estado). Agrupa objetos similares. Unity y Godot recomiendan el *batching* o instancing. Por ejemplo, en Unity activa **SRP Batcher** (URP/HDRP) para reducir trabajo CPU entre draw calls ¹². Usa **GPU Instancing** para muchos objetos idénticos (mismos mesh y material) ¹³. Marca objetos inmóviles como estáticos para **Static Batching** (combina mallas compartiendo material) ¹⁴ ¹⁵. En Godot sugieren combinar mallas estáticas por adelantado (merged meshes) para reducir draw calls, aunque advierten cuidar el *culling* al hacer esto ¹⁶ ¹⁷.
- **Level of Detail (LOD):** usa LOD para reemplazar modelos por versiones más simples a medida que se alejan. Unity explica que “a medida que objetos se alejan, LOD los cambia por mallas más simples para mejorar el rendimiento en GPU” ¹⁸. Planifica varios niveles de detalle (menos polígonos y texturas simplificadas lejos de la cámara) ¹⁸. Solo úsalo donde tenga sentido (escenas con cámara y objetos en movimiento).
- **Occlusion y frustum culling:** evita renderizar lo que no se ve. El *frustum culling* (corte por campo de visión) lo hace casi todo motor automáticamente. Además, activa *Occlusion Culling* para que la cámara ignore objetos ocultos tras otros ¹⁹ ²⁰. En Unity esto se hace «bakeando» la escena y marcando objetos estáticos como ocluidores/ocluibles ²⁰. En el ejemplo MDPI, la

versión optimizada del juego marcó todos los objetos estáticos para occlusion culling, de modo que solo se renderiza lo visible ²¹. Esta técnica reduce el trabajo de GPU (y de CPU) al no dibujar lo que no aparece en pantalla.

- **Reducir efectos costosos:** limita sombras y luces dinámicas. Cada luz dinámica y sombra añade draw calls y cálculos extra. En móviles, es mejor usar iluminación *bakeada*: lightmaps y lightprobes ²² ²³. Desactiva el *shadow casting* donde no sea crítico ²⁴ ²⁵. Usa técnicas sencillas (fake shadows) en vez de físicas complejas.
- **Overdraw (sobredibujo):** dibujar varios objetos transparentes o UI solapados degrada el *fill rate*. Minimiza transparencias superpuestas y partículas excesivas ²⁶. El coste por píxel puede dominar; asegúrate de que los objetos cerrados opacos oculten elementos internos. En móviles y GPUs menos potentes, controlar el *overdraw* es clave.
- **Reutilizar materiales y shaders:** cuantos menos estados cambie la GPU, más rápido. Godot lo explica bien: “**cuantos menos materiales diferentes en escena, más rápido rendereá**” ²⁷. Reutiliza materiales/ texturas iguales en muchos objetos. En el peor caso, agrupa texturas en un atlas para reducir cambios de textura ²⁷. Si creas shaders, procura que compartan código: cada material diferente podría romper un batch. Por ejemplo, en Unity evitar `material` en scripts (hace copias); usar `sharedMaterial` para no romper batches existentes ⁹.
- **Ejemplos prácticos:**
 - En Unity: habilita GPU Instancing marcando *Enable Instancing* en el material; activa el SRP Batcher en el asset de renderizado ¹² ²⁸.
 - En Godot: usa `MultimeshInstance3D` para instanciar muchos objetos con el mismo mesh/ material en una sola draw call.
 - En C++/bajos recursos: agrupar mallas manualmente con APIs (por ejemplo, `Mesh.CombineMeshes()` en Unity) reduce draw calls ²⁹.

Gestión de memoria y assets

- **Texturas y assets:** las texturas suelen ocupar la mayor parte de la VRAM. Reduce resoluciones innecesarias y usa formatos comprimidos (DXT, ASTC, ETC) ³⁰ ³¹. Disminuye el *Max Size* de cada textura al mínimo aceptable visualmente ³². Usa Mipmaps para objetos 3D que cambian de distancia, y desactívalos en sprites/UI estáticos ³³.
- **Atlas y ahorro de draw calls:** agrupa varias texturas en un atlas de sprites cuando sea posible ³⁴. Unity recomienda usar *Sprite Atlas* o herramientas externas para esto ³⁴. Un atlas hace que múltiples sprites se dibujen en un solo draw call.
- **Config. de importación:** desactiva *Read/Write Enabled* para texturas y meshes que no modifiques en tiempo de ejecución ³⁵. Cada vez que esto está activo, duplicas memoria (copia CPU-GPU).
- **Pool de objetos:** como se mencionó antes, minimizar allocs en tiempo de ejecución ayuda a la memoria y al GC. El MDPI confirma que usar pooling mejoró FPS y redujo batches/triángulos, comparado con crear/eliminar enemigos dinámicamente ³⁶ ³⁷. Incluso con más objetos en escena (todos pre-creados), la versión optimizada era más rápida ³⁷.
- **Streaming de contenido:** en mundos grandes, carga niveles/escenas por partes. Godot sugiere unir mallas estáticas “ahead-of-time” en vez de en runtime para evitar overhead (CPU) innecesario ¹⁷. Prepara *levels of detail*, *distant streaming* y *loose loading* de texturas. Esto reduce la RAM usada en cada instante.

Buenas prácticas generales

- **Define plataformas objetivo.** El rendimiento varía según hardware ⁴ ³⁸. Establece unos niveles mínimos (ej. móvil bajo/alto, consola vieja/nueva) y prueba cada configuración. Por

ejemplo, deja detalles gráficos o efectos pesados desactivados en móviles de baja gama, y activa más en hardware potente ³⁸.

- **Itera con pruebas reales.** A veces optimizar en abstracto no ayuda: prueba el juego en hardware real bajo carga (con perfiles de CPU/GPU activos). En móviles, compila *profilers* específicos (FTrace, Perfetto) para monitorear frecuencia de CPU/GPU y consumo de energía durante el juego ³⁹. Deja margen térmico (~35% idle) para evitar *thermal throttling* ⁴⁰.
- **Perfilar, optimizar, volver a perfilar.** Mide repetidamente: cuando cambies el código, perfila de nuevo para confirmar que la optimización funcionó (o no) ⁴¹.
- **Patrones de diseño:** usa técnicas comunes de optimización: *object pooling* (ya visto) ⁴², *spatial partitioning* (quadrees, octrees) para físicas y render, *Level Streaming*, *Data-Oriented Design* (acceso lineal a arrays, por ejemplo).
- **Documentación oficial:** lee las guías de tu motor. Unity, Unreal y Godot tienen secciones dedicadas a optimización (por ejemplo, Unity ofrece guías detalladas de *Batching*, *Profiling* y *Mobile Optimization* ⁴³ ²⁹). Epic Games (Unreal) y GPUOpen (AMD) publican consejos de perfilado avanzados ⁵.

En resumen, para un juego fluido hay que **medir primero**, luego **aliviar la CPU reduciendo lógicas innecesarias** (menos cálculos, pooling, multihilo) y **aligerar la GPU minimizando draw calls y cargas gráficas** (batching, instancing, LOD, culling, texturas optimizadas) ¹² ²⁷. No olvides también la gestión de memoria (compresión, atlas, memoria caché) y perfilar en todas las etapas ⁴⁴ ¹¹. Aplicando estas técnicas y validando con perfiles, tu juego ganará fluidez y estabilidad en cualquier plataforma.

Fuentes: Guías oficiales de Unity y Godot sobre rendimiento ¹ ²⁹ ⁸ ²⁷; artículos técnicos (por ejemplo, Unity, AMD, MDPI) sobre *batching*, *culling*, *profiling* y *object pooling* ¹² ⁵ ³⁶ ⁴². Estos recursos respaldan cada recomendación aquí expuesta.

¹ ³⁸ ³⁹ ⁴⁰ ⁴⁴ Prácticas recomendadas para perfilar el rendimiento del juego | Unity

<https://unity.com/es/how-to/best-practices-for-profiling-game-performance>

² ³ ⁴ Best practices for profiling game performance | Unity

<https://unity.com/how-to/best-practices-for-profiling-game-performance>

⁵ Unreal Engine Performance Guide - AMD GPUOpen

<https://gpuopen.com/learn/unreal-engine-performance-guide/>

⁶ ⁷ ⁹ ¹² ¹³ ¹⁴ ²⁶ Managing GPU usage for PC and console games | Unity

<https://unity.com/how-to/gpu-optimization>

⁸ ¹¹ ⁴¹ Optimización de CPU — Documentación de Godot Engine (4.5) en español

https://docs.godotengine.org/es/4.5/tutorials/performance/cpu_optimization.html

¹⁰ ²¹ ³⁶ ³⁷ ⁴² Improving Mobile Game Performance with Basic Optimization Techniques in Unity

<https://www.mdpi.com/2673-3951/3/2/14>

¹⁵ ²² ²³ ²⁴ ²⁵ ²⁸ ⁴³ Consejos de optimización del arte para desarrolladores de juegos móviles, parte 2

<https://unity.com/es/how-to/mobile-game-optimization-tips-part-2>

¹⁶ ¹⁷ ²⁷ Optimización de GPU — Documentación de Godot Engine (4.x) en español

https://docs.godotengine.org/es/4.x/tutorials/performance/gpu_optimization.html

¹⁸ ¹⁹ ²⁰ ²⁹ ³⁰ ³¹ ³² ³³ ³⁴ ³⁵ Art optimization tips for mobile game developers part 1

<https://unity.com/how-to/mobile-game-optimization-tips-part-1>